

Lec - 32

✓ `isSelected()` in Selenium

The `isSelected()` method in Selenium is used to **check the selection state** of web elements such as:

- **Checkboxes**
- **Radio buttons**
- `<option>` tags within `<select>` dropdowns (in multi-select)

It returns a **boolean value**:

- `true` if the element is selected (checked),
- `false` if it's not selected.

Syntax:

```
boolean status = element.isSelected();
```

Example 1: Working with a Checkbox

```
WebDriver driver = new ChromeDriver();  
driver.get("https://example.com");  
  
// Locate the checkbox
```

```
WebElement termsCheckbox = driver.findElement(By.id("agreeTerms"));

// Check if it's selected
if (!termsCheckbox.isSelected()) {
    termsCheckbox.click(); // select if not already selected
}

System.out.println("Checkbox selected: " + termsCheckbox.isSelected());

driver.quit();
```

```
driver.get("https://omayo.blogspot.com/");

WebElement blueCheckbox = driver.findElement(By.id("checkbox2"));
boolean selectedStatus = blueCheckbox.isSelected();

if(selectedStatus) {

    System.out.println("Checkbox is in selected state");

} else {

    System.out.println("Checkbox is not in selected state");

}

driver.quit();
```

Example 2: Radio Button

```
WebElement maleRadio = driver.findElement(By.id("genderMale"));
WebElement femaleRadio = driver.findElement(By.id("genderFemale"));

System.out.println("Is Male selected? " + maleRadio.isSelected());
System.out.println("Is Female selected? " + femaleRadio.isSelected());
```

//Selecting a radio button with value as “Bike”. (By.xpath("//input[@value='Bike']")

(Line-2 completion in below image)

```
driver.get("https://omayo.blogspot.com/");

WebElement bikeRadioButton = driver.findElement(By.xpath("//input[@value='Bike']"));
boolean selectedStatus = bikeRadioButton.isSelected();
if(selectedStatus) {
    System.out.println("Checkbox is in selected state");
} else {
    System.out.println("Checkbox is not in selected state");
}

driver.quit();
```

Example 3: <option> in <select> dropdown

```
Select select = new Select(driver.findElement(By.id("dropdown")));
List<WebElement> options = select.getOptions();

for (WebElement option : options) {
    if (option.isSelected()) {
        System.out.println("Selected option: " + option.getText());
    }
}
```

Notes:

- `isSelected()` works only on elements where selection state is meaningful.
- It does **not** apply to regular buttons or text fields.
- For better synchronization, you can use **explicit waits** before calling `isSelected()`.

`navigate()` in Selenium WebDriver

The `navigate()` method in Selenium WebDriver belongs to the `WebDriver.Navigation` interface. It is used to **control browser navigation**, allowing you to simulate user actions like:

- Visiting a URL
- Moving forward and backward in browser history
- Refreshing the current page

Commonly Used Methods under `navigate()`

1. `navigate().to(String url)`

- Opens the specified URL (similar to `driver.get(url)`).
- Can be used to open a new web page.

```
driver.navigate().to("https://example.com");
```

2. `navigate().back()`

- Simulates pressing the browser's **back button**.

- Useful for testing navigation between pages.

```
driver.navigate().back();
```

3. `navigate().forward()`

- Simulates pressing the browser's **forward button** after going back.

```
driver.navigate().forward();
```

4. `navigate().refresh()`

- Reloads the current web page.

```
driver.navigate().refresh();
```

vs `navigate().to()` vs `driver.get()`

Feature	<code>driver.get()</code>	<code>navigate().to()</code>
Purpose	Loads a URL	Loads a URL
Waits	Waits for the entire page to load	Also waits, but more flexible in chaining with other navigation
Usage	Simpler for one-time navigation	Preferred when chaining back/forward/refresh

Example of All Navigate Methods

```
WebDriver driver = new ChromeDriver();

// Open a webpage
driver.navigate().to("https://example.com");

// Go to another page
driver.navigate().to("https://example.com/page2");

// Go back to previous page
driver.navigate().back();

// Go forward again
driver.navigate().forward();

// Refresh the page
driver.navigate().refresh();

driver.quit();
```

When to Use `navigate()`?

Use `navigate()` when:

- You are moving back and forth between pages.
- You want to simulate user-like browser behavior.
- You are chaining page visits with refresh or back/forward controls.



getPageSource() in Selenium WebDriver

The `getPageSource()` method in Selenium is used to **retrieve the entire HTML source code** of the current web page as a `String`.



Syntax

```
String pageHTML = driver.getPageSource();
```

- `driver` is an instance of `WebDriver`.
 - `getPageSource()` returns the **HTML markup** of the current page.
-



What Does It Return?

It returns **all the HTML** that the browser has loaded for the current page—including:

- All elements in the DOM
 - Inline JavaScript and CSS
 - Any dynamically inserted elements (only if they are present in the DOM at the time of the call)
 - ♦ However, it does **not** include content rendered dynamically by JavaScript **after** the page has loaded (e.g., content loaded via AJAX unless it's in the DOM already).
-



Example Usage

```
WebDriver driver = new ChromeDriver();  
driver.get("https://example.com");  
  
String html = driver.getPageSource();  
System.out.println(html);  
  
driver.quit();
```

This will print the full HTML source of the loaded page to the console.

Use Cases

1. **Debugging:** Check if a certain element is present in the page's HTML.
2. **Validation:** Verify content that isn't easily captured through UI elements.
3. **Text Search:** Search for a word or string in the HTML.
4. **Page Analysis:** Use in scraping or text extraction tasks (though Selenium isn't ideal for full scraping).

Assertions in Testing:

```
Assert.assertTrue(driver.getPageSource().contains("Welcome User"));
```

What It Cannot Do

- It does **not** show the rendered UI or JavaScript execution results if they are not in the DOM.
- It cannot be used to interact with elements—you must still use locators like `findElement()` for interaction.

Lec - 33

♦ 1. `getTagName()`

Purpose:

Returns the **tag name** of a web element, like `input`, `button`, `div`, etc.

Syntax:

```
String tag = element.getTagName();
```

Example:

```
WebElement inputField = driver.findElement(By.id("username"));

System.out.println("Tag Name: " + inputField.getTagName()); // Output:
input
```

Use Case:

- To confirm you are interacting with the correct type of HTML element.
- Helpful in reusable methods when validating dynamic elements.

♦ 2. `getCssValue(String propertyName)`

Purpose:

Fetches the **computed CSS value** of a given property (like `color`, `font-size`, `background-color`) for an element.

Syntax:

```
String value = element.getCssValue("property-name");
```

Example:

```
WebElement button = driver.findElement(By.id("submit"));

String backgroundColor = button.getCssValue("background-color");

System.out.println("Background Color: " + backgroundColor);
```

Use Case:

- Useful for **UI testing**, such as checking that a button is styled correctly (color, margin, etc.).
- Helps validate dynamic styles (e.g., hover effects, disabled states).

♦ 3. `getSize()`

Purpose:

Returns the **dimension** (width and height) of the element as a `Dimension` object.

Syntax:

```
Dimension size = element.getSize();

int height = size.getHeight();

int width = size.getWidth();
```

Example:

```
WebElement logo = driver.findElement(By.id("logo"));
Dimension size = logo.getSize();
System.out.println("Height: " + size.getHeight());
System.out.println("Width: " + size.getWidth());
```

Use Case:

- Verifying **responsive design** and layout.
 - Confirming the visibility or proper rendering of an element on different screen sizes.
-

◆ 4. **getRect()**

Purpose:

Returns a **Rectangle** object containing the **height, width, X and Y coordinates** of the element.

Syntax:

```
Rectangle rect = element.getRect();
int width = rect.getWidth();
int height = rect.getHeight();
int x = rect.getX();
int y = rect.getY();
```

Example:

```
WebElement banner = driver.findElement(By.id("mainBanner"));

Rectangle rect = banner.getRect();

System.out.println("X: " + rect.getX());

System.out.println("Y: " + rect.getY());

System.out.println("Height: " + rect.getHeight());

System.out.println("Width: " + rect.getWidth());
```

Use Case:

- For **pixel-level layout verification** (e.g., ensuring two elements are aligned).
- Checking that an element isn't pushed off-screen or overlapping another.

Summary Table:

Method	Returns	Use For
<code>getTagName()</code>	Element tag name (<code>div</code> , <code>input</code>)	Element type checking
<code>getCssValue()</code>	Computed CSS property value	Style validation
<code>getSize()</code>	Width & height (as <code>Dimension</code>)	Size verification
<code>getRect()</code>	X, Y, width, height (<code>Rectangle</code>)	Layout & position validation

Lec - 34

1. `fullScreen()`

Purpose:

- Maximizes the browser **to full screen mode**, similar to pressing **F11**.
- Available from Selenium 4 onwards.

Syntax:

```
driver.manage().window().fullscreen();
```

Behavior:

- Unlike `maximize()`, which only enlarges the window to the maximum desktop size, `fullscreen()` actually puts the browser into full-screen mode (covering the taskbar and tabs).

Example:

```
WebDriver driver = new ChromeDriver();  
driver.get("https://example.com");  
driver.manage().window().fullscreen();
```

Use Case:

- Useful when testing **kiosk mode applications** or checking UI responsiveness in **true full screen**.
-

2. **setSize(Dimension targetSize)**

Purpose:

- Manually sets the **width and height** of the browser window.

Syntax:

```
driver.manage().window().setSize(new Dimension(width, height));
```

Example:

```
Dimension d = new Dimension(800, 600);  
driver.manage().window().setSize(d);
```

Use Case:

- Useful in **responsive testing**, simulating different screen sizes (mobile, tablet, desktop).
- Automating visual layout tests.

 To use it: `import org.openqa.selenium.Dimension;`

3. **getClass()** (Java method)

Purpose:

- Comes from the base class `Object` in Java.
- Returns the **runtime class** of the object.

✓ Syntax:

```
Object obj = driver.findElement(By.id("someId"));
System.out.println(obj.getClass());
```

🔍 Behavior:

- Returns a `Class` object representing the runtime type (e.g., `class org.openqa.selenium.remote.RemoteWebElement`).

📌 Use Case:

- Mainly used for **debugging** and **reflection**.
- Helps you understand what kind of object you are dealing with, especially when working with interfaces or generic types.

🕒 4. `pageLoadTimeout(Duration time)`

📘 Purpose:

- Defines the **maximum amount of time WebDriver will wait** for a page to load completely.

✓ Syntax (Selenium 4+):

```
driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(20));
```

 If the page doesn't load within the timeout, a `TimeoutException` is thrown.

 Example:

```
import java.time.Duration;

driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(10));

driver.get("https://example.com");
```

 Use Case:

- Useful for handling **slow-loading pages** or unstable internet conditions.
- Prevents your tests from hanging forever.

 Summary Table

Method	Purpose	Returns/Effect	Use Case
<code>fullscreen()</code>	Full screen browser like F11	Window expands to full screen	Kiosk mode, visual testing
<code>setSize(Dimension)</code>	Resize browser to fixed width × height	Sets browser size	Responsive/mobile UI testing
<code>getClass()</code> (Java)	Get runtime class of an object	<code>Class<?></code> object	Debugging, dynamic type checking
<code>pageLoadTimeout()</code>	Set max wait time for page load	Throws <code>TimeoutException</code> on fail	Timeout control for slow pages